INSTITUTO MILITAR DE ENGENHARIA (IME)

Seção de Engenharia Elétrica — SE/3

Programação Aplicada à Engenharia Elétrica (SE/3 - Eletrônica)

Manual Mestre de Execução e Aprendizado: Do Zero ao Deploy

Montagem Física, Ambiente Virtual, Pastas Compartilhadas, WinSCP, POO C++, IIO e Cross-Compilação

Instituição:	Instituto Militar de Engenharia (IME) / SE/3
Missão do Projeto:	Monitoramento Inteligente de Carga Sensível em Operações Logísticas
Kit de Hardware:	STMicroelectronics **STM32MP157D-DK1** (Linux Embarcado OpenSTLinux)
Sensor Alocado (Nº 3):	Módulo de Obstáculo Infravermelho Reflexivo Digital (**HW-201** / LM393)
Critério de Avaliação:	Projeto = 4.000 pontos (VF) em 5 entregas de 800 pts | Listas = 6.000 pontos
Prazo Entrega 1:	**26/10/2026 às 23:59** — Classe C++ com Doxygen no Git (800 pontos)
Prazo Entrega 2:	**02/11/2026 às 23:59** — Binário ARM Funcional no Kit + README (800 pontos)

1 Apresentação Geral e Novo Critério Oficial de Notas

Em comunicado recente divulgado no Google Classroom da disciplina (Figura 1), foram estabelecidas as regras definitivas para o cálculo das notas da disciplina:

- **Escala Global:** Cada aluno pode somar até **10.000 pontos** na matéria (para evitar dízimas e facilitar pontuações decimais).
- **O Trabalho (Projeto Final):** Vale **4.000 pontos** divididos em 5 entregas parciais de **800 pontos** cada. O projeto possui o peso equivalente à **Verificação Final (VF)**.
- **As Listas de Exercícios:** Somam um total de **6.000 pontos** distribuídos entre si, equivalendo à **Verificação Corrente (VC) e Verificações Especiais (VEs)**.

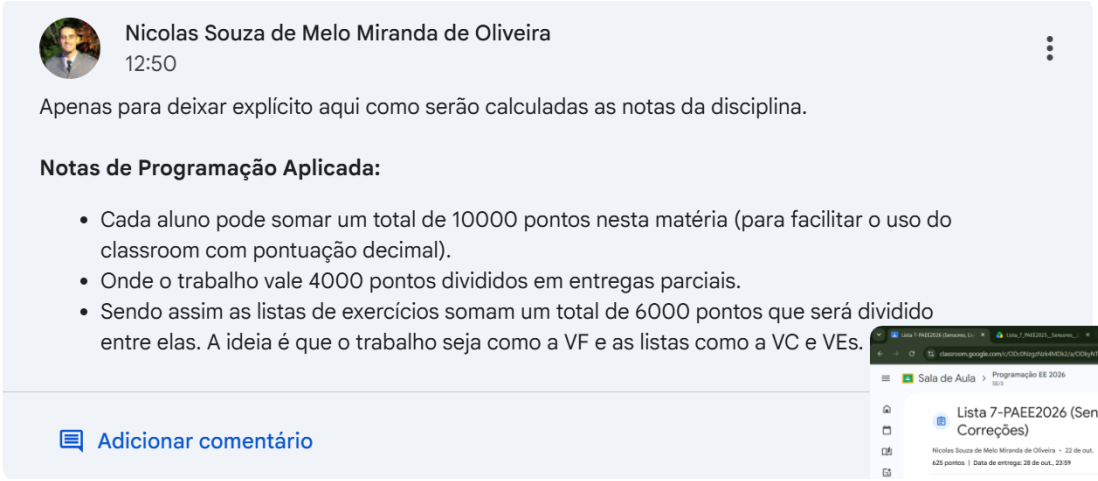


Figura 1: Comunicado oficial no Google Classroom detalhando o sistema de pontuação (10.000 pts total; 4.000 pts Projeto; 6.000 pts Listas).

Etapa	Prazo Final	Pontuação	Entregável Principal
Entrega 0	19/10/2026	Concluída	Validação física da montagem com o kit desligado.
Entrega 1	**26/10/2026**	**800 pts**	Classe C++ (`IrObstacleSensor`) com Doxygen no Git. **Deve rodar na placa!**
Entrega 2	**02/11/2026**	**800 pts**	Binário ARM funcional na STM32MP1 + Instruções no README.md.
Entrega 3	09/11/2026	**800 pts**	Telemetria UDP contínua (porta 5000) e protocolo documentado.
Entrega 4	16/11/2026	**800 pts**	Interface gráfica (GUI) no computador com gráfico e exportação CSV.
Entrega 5	16/11/2026	**800 pts**	**Mesma data da E4!** Vídeo de demonstração e apresentação final.

Critério Eliminatório de Avaliação: Regra de POO Estrita

O edital oficial determina: **é expressamente proibida a utilização de variáveis globais e de funções soltas fora de classes**.

- **A razão de engenharia:** Na fase final, os 11 grupos da turma integrarão seus módulos no servidor central. Variáveis globais ou funções livres causariam conflito de nomes no linker (*symbol collision*).
- **Padrão exigido:** Todos os estados do sensor (caminho de arquivo, último valor, limiares) devem ser atributos `private`, manipulados unicamente por métodos `public` bem documentados.

2 O Hardware: Placa STM32MP157D-DK1 e Sensor HW-201

Antes de manusear os componentes, é indispensável conhecer as características elétricas e arquiteturais dos equipamentos fornecidos:



Figura 2: Identificação oficial do kit: Caixa do kit Discovery com etiqueta de conferência (`KIT STM32MP1 DK1 + CABO USB C + SD CARD`) e etiqueta lateral do fabricante (`STM32MP157D-DK1`).

2.1 A Placa: STMicroelectronics STM32MP157D-DK1

Não se trata de um microcontrolador comum (como Arduino ou STM32F1/F4), mas de um **Microprocessador (MPU) Híbrido**:

- **Dual Arm Cortex-A7 @ 800 MHz:** Processador principal de 32 bits que roda um **Sistema Operacional Linux Completo** gravado no cartão microSD. É nesse ambiente Linux em espaço de usuário que a sua aplicação C++ vai executar.
- **Arm Cortex-M4 @ 209 MHz:** Coprocessador interno para tarefas de tempo real crítico.
- **Nível Lógico de 3,3 V:** Todos os barramentos de entrada/saída (GPIOs e conversores A/D) operam estritamente em **3,3 V**.

2.2 O Sensor: Módulo Infravermelho Reflexivo (HW-201 / LM393)

O Sensor Nº 3 atribuído ao grupo tem a finalidade de detectar aproximação ou abertura de compartimento através da interrupção/reflexão de luz infravermelha:

Como o Sensor HW-201 Funciona Fisicamente?

Na extremidade frontal da plaquinha encontram-se dois elementos ópticos de 5 mm:

1. **Emissor IR (transparente):** Emite continuamente feixes de luz infravermelha.
2. **Fototransistor Receptor (preto):** Recebe os feixes de luz que batem em um obstáculo próximo (a tampa do compartimento) e refletem de volta.
3. **Comparador LM393 e Trimpot Azul:** O circuito integrado LM393 compara o sinal do receptor óptico com a tensão de referência ajustada pelo parafuso azul (trimpot). O circuito opera em lógica **Active-Low**:
   - **Tampa Fechada (Reflexão Presente):** O pino de saída `OUT` vai para **Nível Lógico BAIXO (0 V)** e o LED indicador da placa acende.
   - **Tampa Aberta / Violação (Sem Reflexão):** O feixe infravermelho se propaga livremente, o pino `OUT` vai para **Nível Lógico ALTO (3,3 V)** e o LED indicador apaga.

Regras Críticas de Segurança de Bancada

1. **Proibição Absoluta do Pino 5V:** O sensor HW-201 deve ser alimentado **exclusivamente em 3,3 V**. Como a saída `OUT` acompanha a tensão de alimentação, ligar em 5V injetaria 5V no conversor da placa e causaria **queima permanente** do chip STM32MP1.
2. **Preservação do Cartão microSD:** O cartão inserido na placa contém a imagem oficial do Linux configurada para a disciplina. Nunca o remova da placa.
3. **Montagem Totalmente Desenergizada:** É obrigatório desconectar todos os cabos (USB-C, Micro-USB e rede) antes de espetar qualquer jumper.

3 Passo 0: A Montagem Física e o Repositório Git (Entrega 0)

3.1 Mapeamento dos Jumpers Pino a Pino

Para realizar a ligação segura, utilize **3 cabos jumper Fêmea-Macho**. Posicione a placa virada de costas (lado com adesivo branco `USER: root SENHA: root` para cima), deixando os conectores pretos fêmea voltados para você e o slot de cartão microSD apontando para a esquerda:

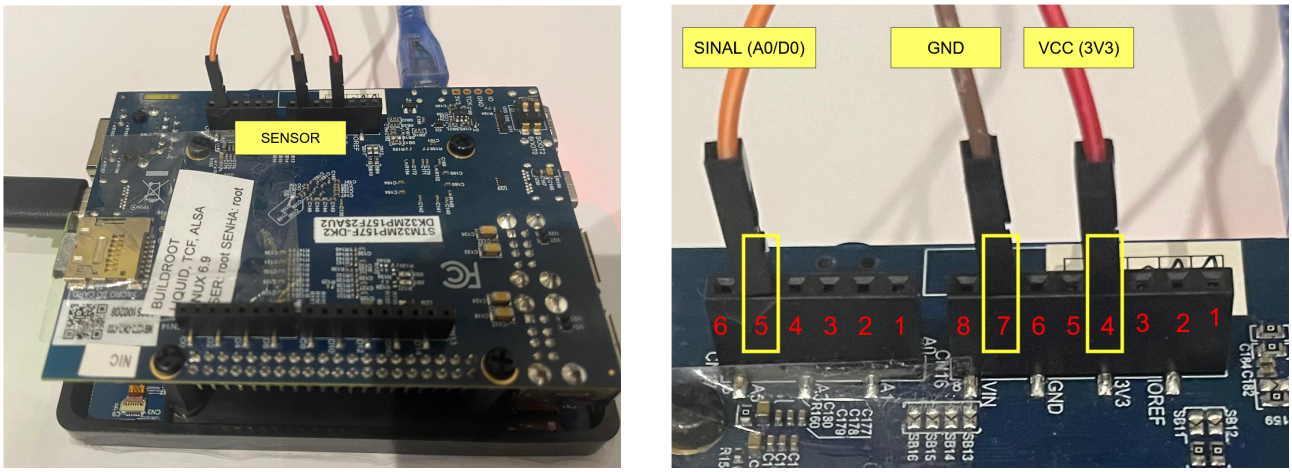


Figura 3: Referência oficial de conexão física (Figuras 4 e 5 do edital): Conexão da alimentação e do sinal analógico contando os furos rigorosamente **da direita para a esquerda**.

Pino no Sensor	Cor Sugerida	Conector na STM32MP1	Posição Exata	Conferência
`VCC`	Vermelho / Cinza	Bloco da Direita (8 furos)	**Pino 4** (acima de `3V3`)	Sobram 3 furos dele.
`GND`	Preto / Branco	Bloco da Direita (8 furos)	**Pino 7** (acima de `GND`)	Sobram 2 furo de 3,3V e 1 f querda (`VIN`).
`OUT`	Laranja / Preto	Bloco da Esquerda (6 furos)	**Pino 5** (Entrada ADC)	Sobram 4 furos dele e 1 furo esquerda.

3.2 Configuração do Repositório Git e Submissão

A entrega de todas as etapas é realizada via **link do commit final no GitHub**:

```
# 1. No diretorio do projeto, inicialize o git caso ainda nao o tenha feito
cd c:/Users/jvpar/estudo/3.2/ProgAplicada/projeto_final
git init

# 2. Crie o arquivo .gitignore para evitar comitar arquivos binarios ou
   pesados
cat << 'EOF' > .gitignore
*.exe
*.o
app_sensor
app_sensor_arm
*.aux
*.log
*.toc
*.out
.vscode/
```

```
EOF

# 3. Adicione os arquivos do projeto e faca o primeiro commit
git add include/ src/ Makefile Doxyfile README.md .gitignore docs/
git commit -m "Entrega 1: Classe C++ IrObstacleSensor documentada com
    Doxygen"

# 4. Vincule ao repositorio oficial no GitHub e envie:
git remote add origin https://github.com/VPariz/stm32-projeto-final-prog-
    aplicada.git
git branch -M main
git push -u origin main
```

4 Passo 1: Subsistema IIO, POO e Doxygen (Entrega 1 — 800 Pontos)

A **Entrega 1** (Figura 4) possui prazo em **26/10/2026 às 23:59** e pontuação de **800 pontos**.

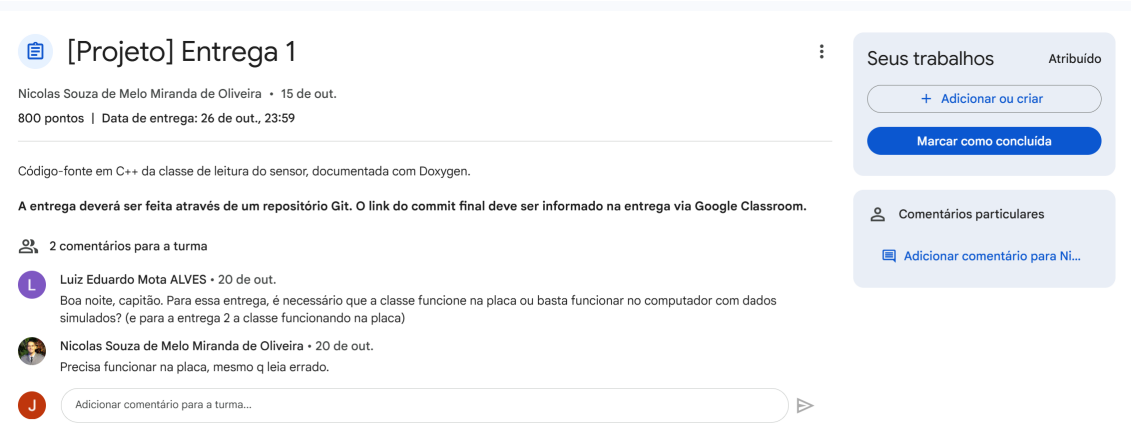


Figura 4: Atividade da Entrega 1 no Google Classroom com esclarecimento crítico do corpo docente nos comentários.

Aviso Crítico do Classroom: Exigência de Execução na Placa

Na aba de comentários da Entrega 1, um aluno questionou expressamente:

> *"Para essa entrega, é necessário que a classe funcione na placa ou basta funcionar no computador com dados simulados?"*

A resposta oficial do docente foi categórica:

> **"Precisa funcionar na placa, mesmo q leia errado."**

Conclusão de Engenharia: A classe em C++ não pode ser apenas um protótipo para PC. Ela deve acessar o nó de arquivo virtual do Kernel Linux (`/sys/bus/iio/...`), ser compilada (ou transferida) para a placa e executar uma leitura real do conversor na STM32MP157D-DK1, mesmo que a calibração final do limiar ocorra na Entrega 2.

4.1 O Subsistema IIO no Linux (Industrial I/O)

No Linux da placa STM32MP157D-DK1, o conversor A/D é exposto como arquivo virtual no diretório:

`/sys/bus/iio/devices/iio:device0/`

Dentro dele residem arquivos como `in_voltage0_raw`, `in_voltage1_raw`, etc. Ler esse arquivo equivale a solicitar ao Kernel a leitura analógica imediata no conversor físico.

4.2 Procedimento Presencial: Descoberta do Canal Exato do ADC

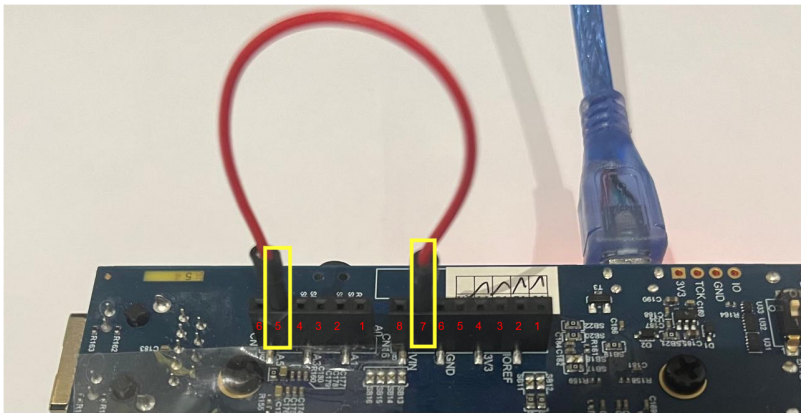


Figura 5: Identificação de canal (Figura 3 do edital): Conexão direta do pino ADC ao GND com jumper para identificação de leitura zero.

Roteiro de 5 Minutos no Laboratório

1. Com a placa desligada, coloque um jumper conectando o **Pino 5 do bloco de 6 furos (ADC)** ao **Pino 7 (GND)** do bloco de alimentação.
2. Ligue a placa com o cabo USB-C na porta `PWR_IN` e conecte o cabo Micro-USB serial no PC.
3. Abra o TeraTerm configurado para **115200 8N1**, aperte Enter e faça login com usuário `root` e senha `root`.
4. No terminal da placa, digite:

```
cd /sys/bus/iio/devices/iio:device0/
cat in_voltage*_raw
```

5. O arquivo `in_voltageX_raw` que retornar **0** (ou ruído elétrico desprezível, ex.: 1) é o canal do pino testado.
6. Desligue, reconecte o sensor HW-201 e confira que com a tampa fechada (reflexão) o valor lido é baixo, e com o feixe desimpedido o valor é alto.

4.3 Código da Classe: `IrObstacleSensor.hpp` e `IrObstacleSensor.cpp`

```
// Joao Victor Pariz - Projeto Final - Sensor HW-201 - SE/3 - Eletronica -
   Programacao Aplicada
#ifndef IR_OBSTACLE_SENSOR_HPP
#define IR_OBSTACLE_SENSOR_HPP

#include <string>
#include <fstream>
#include <stdexcept>
#include <sstream>
#include <iomanip>

/**
 * @file IrObstacleSensor.hpp
 * @brief Classe de leitura e diagnostico do Sensor HW-201 via Industrial I/
    O (IIO).
 */
```

```
class IrObstacleSensor {
private:
    std::string devicePath;      /**< Caminho no sysfs do Linux ou mock para
        PC */
    int rawThreshold;            /**< Limiar de comutacao entre estado
       seguro e violado */
    int lastRawValue;            /**< Ultimo valor bruto inteiro lido do ADC
        */
    double referenceVoltage;     /**< Tensao de referencia eletrica (3.3 V)
       */
    int maxAdcResolution;        /**< Resolucao maxima da escala do ADC
       (4095 para 12 bits) */

public:
    explicit IrObstacleSensor(
        const std::string& path = "/sys/bus/iio/devices/iio:device0/
           in_voltage0_raw",
        int threshold = 2000,
        double vref = 3.3,
        int maxAdc = 4095
    );

    virtual ~IrObstacleSensor() = default;

    int readRaw();                     /**< Efetua a leitura atomica do
       subsistema IIO */
    double getVoltage() const;         /**< Converte valor inteiro em tensao
        (V) */
    bool isObstacleDetected() const;   /**< Retorna true se houver reflexao
       (Active-Low) */
    bool isCompartmentViolated() const;/**< Retorna true se a carga foi
       aberta */
    std::string getFormattedStatus() const;
    int getLastRawValue() const;
    void setThreshold(int newThreshold);
};

#endif // IR_OBSTACLE_SENSOR_HPP
```

4.4 Documentação Doxygen e Checklist da Entrega 1

1. Execute `doxygen Doxyfile` na pasta raiz do projeto. A documentação será compilada na pasta `docs/doxygen/html/`.

2. Abra o arquivo `docs/doxygen/html/index.html` no navegador e comprove que a classe `IrObstacleSensor` e todos os seus métodos públicos e privados estão documentados.

3. Teste a compilação cruzada ou nativa da aplicação:

```
# Compilacao nativa no PC com simulador
g++ -Wall -Wextra -std=c++17 -Iinclude src/IrObstacleSensor.cpp src/main.
    cpp -o app_sensor
./app_sensor mock_sensor.txt 127.0.0.1 5000 5
```

4. Realize o commit no Git com a tag ou mensagem da Entrega 1 e submeta o link do commit no Google Classroom.

5 Passo 2: Compilação Cruzada, Deploy e Execução (Entrega 2 — 800 Pontos)

A **Entrega 2** (Figura 6) possui prazo em **02/11/2026 às 23:59** e pontuação de **800 pontos**.

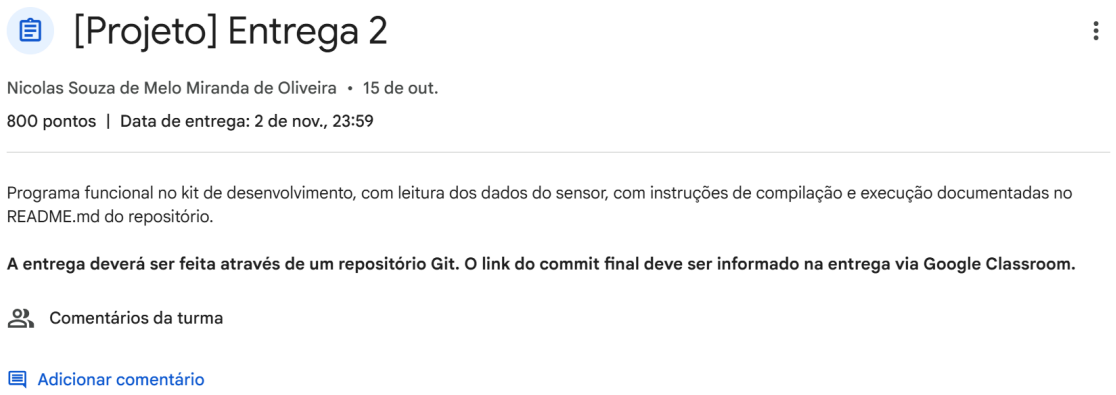


Figura 6: Atividade da Entrega 2 no Google Classroom: Programa funcional no kit STM32MP157D-DK1 com documentação no README.md.

5.1 Fundamentos da Cross-Compilação e Deploy com WinSCP

Como o computador possui processador x86_64 e a placa possui um processador Arm Cortex-A7 de 32 bits, usamos a toolchain para gerar o binário `app_sensor_arm`:

```
# Na VM Linux com a toolchain instalada:
make cross
# O executavel 'app_sensor_arm' sera gerado na pasta
```

5.2 Transferência para a Placa e Calibração do Trimpot

1. No WinSCP, conecte no IP `192.168.42.2` (usuário `root`, senha `root`) e arraste `app_sensor_arm` para `/home/root/`.
2. No terminal da placa via SSH (`ssh root@192.168.42.2`) ou serial:

   ```
   chmod +x app_sensor_arm
   ./app_sensor_arm
   ```

3. Ajuste a sensibilidade do sensor girando o trimpot azul com uma chave de fenda até que a reflexão com a tampa a 2 cm comute o valor de leitura.
4. Tire prints do terminal comprovando a alternância entre `[STATUS: SEGURO]` e `[STATUS: VIOLADO]` e documente no `README.md`.

6 Passo 3: Telemetria via Socket UDP (Entrega 3 — 800 Pontos)

A **Entrega 3** (Figura 7) possui prazo em **09/11/2026 às 23:59** e pontuação de **800 pontos**.

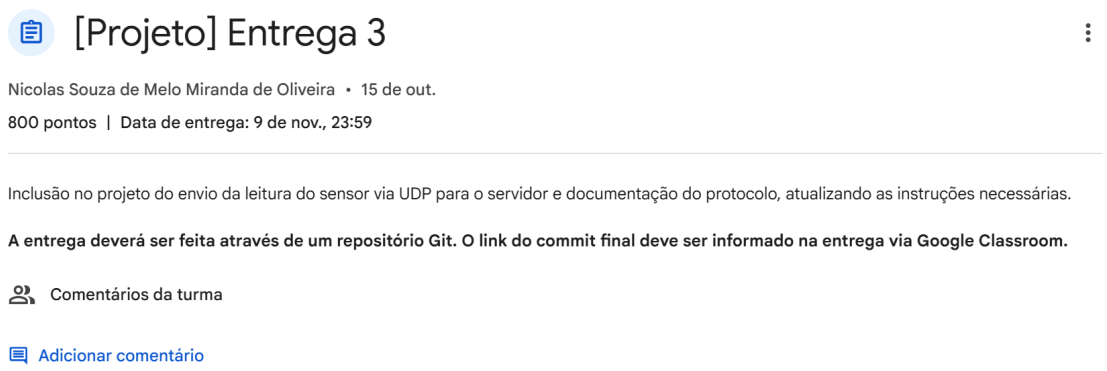


Figura 7: Atividade da Entrega 3: Transmissão da leitura do sensor via pacotes UDP para o servidor com protocolo documentado.

6.1 A Classe `UdpTransmitter` e o Protocolo de Rede

A classe `UdpTransmitter` encapsula sockets POSIX/WinSock, transmitindo datagramas UDP para a porta 5000 do computador central. O formato JSON transmitido é:

```
{
  "sensor_id": "G3_IR_HW201",
  "raw_value": 3840,
  "voltage_v": 3.09,
  "is_violated": true,
  "status_text": "VIOLADO",
  "timestamp_ms": 1729801234567
}
```

7 Passos 4 e 5: Interface Gráfica, Vídeo e Apresentação (800 + 800 Pontos)

As **Entregas 4 e 5** (Figuras 8 e 9) compartilham a data final de entrega: **16/11/2026 às 23:59**.

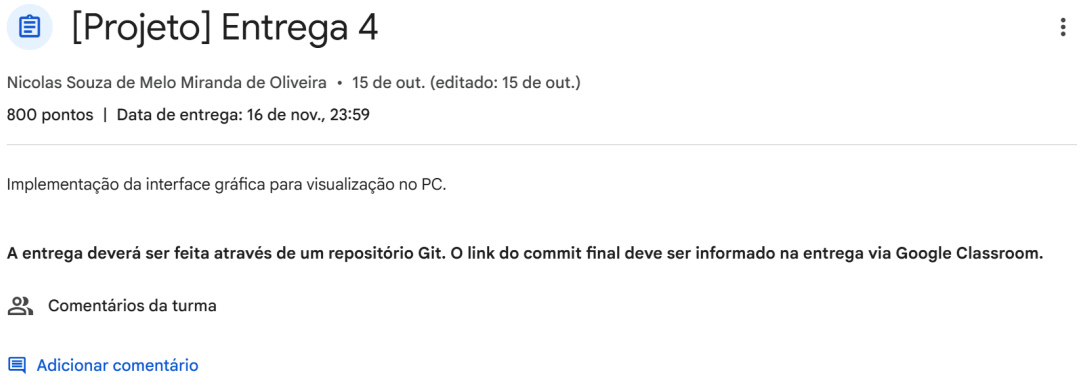


Figura 8: Atividade da Entrega 4: Interface gráfica de supervisão no PC.

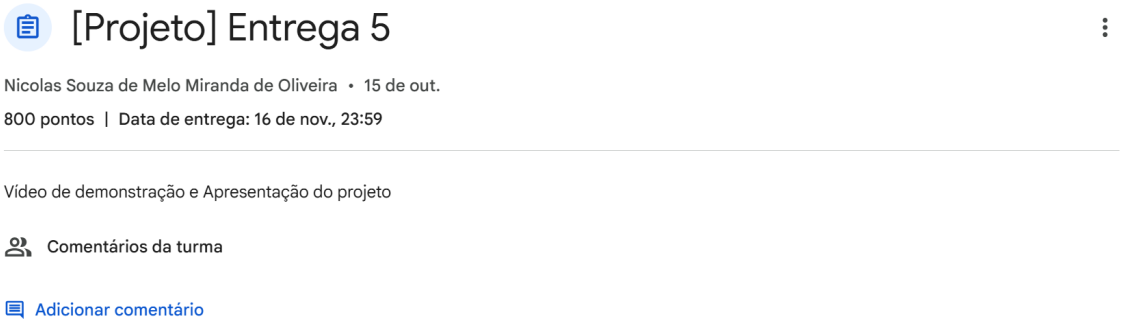


Figura 9: Atividade da Entrega 5: Apresentação e vídeo de demonstração do sistema integrado.

7.1 Dashboard Gráfico em Python (`scripts/gui_dashboard.py`)

O script de interface gráfica escuta na porta UDP 5000 e plota um gráfico deslizante dos últimos 60 segundos com Matplotlib/Tkinter, gerando alarme sonoro/visual em caso de violação e salvando o log contínuo em CSV.

8 Matriz Consolidada de Entregáveis: O que está Pronto vs. O que Falta Fazer

A tabela a seguir apresenta o status minucioso de cada componente do Projeto Final:

Etapa	Itens Exigidos e Ações Necessárias	Status Atual
Entrega 0	Conexão física com placa desligada; fotos dos jumpers de 3,3V, GND e ADC; repositório GitHub criado com integrantes.	**CONCLUÍDO**
Entrega 1	✓ Código em C++ da classe `IrObstacleSensor` com POO estrita. ✓ Documentação Doxygen completa em `docs/doxygen/html/`. ☐ Executar `git init` em `projeto_final/`, comitar e enviar para o GitHub. ☐ Submeter a URL do commit final no Google Classroom (Prazo: 26/10). ☐ Garantir teste presencial na placa lendo o nó do IIO.	**PRONTO P/ ENVIO**
Entrega 2	✓ Código funcional pronto para compilação ARM. ☐ Compilação cruzada (`make cross`) gerando `app_sensor_arm`. ☐ Transferir via WinSCP para a placa e calibrar trimpot do HW-201. ☐ Prints da execução no terminal documentados no `README.md`.	**CÓDIGO PRONTO**
Entrega 3	✓ Classe `UdpTransmitter` implementada em C++. ✓ Scripts de teste de recepção UDP em Python implementados. ☐ Teste do fluxo UDP completo placa → PC e commit final.	**CÓDIGO PRONTO**
Entrega 4	✓ Interface gráfica em Python (`gui_dashboard.py`) pronta com gráfico temporal e exportação CSV. ☐ Capturas da tela do dashboard operando com dados reais.	**CÓDIGO PRONTO**
Entrega 5	☐ Gravação do vídeo de demonstração (3 a 5 min) mostrando o sensor sendo acionado e o alerta no PC. ☐ Apresentação final do projeto.	**ROTEIRO PRONTO**